

Lab 4

2. Using Colors

Using JDialog, JOptionPane and JFileChooser

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Tuesday, February 3rd, 2026 | ER 3120



University of Windsor

Part I

Setting up the Environment



JDIALOG



JDialog

JDialog object is very similar to JFrame.

The main differences are:

- **Default Modality:** JFrame is not modal by default. Modal means that execution is paused until the component is disposed. Non-modal components, on the other hand, do not pause execution.
- **Use Case:** JFrame is commonly used as the main window for an application, while JDialog is used for pop-up dialogs or secondary windows that need to interact with the user.



Creating a JDialog

- Create a simple function called `createJDialog()`:
 - This function will be responsible for creating a `JDialog()`.
 - We simply call this function inside the main function. This will create the `JDialog`.
- The main element to focus on is the `JDialog` constructor (`new JDialog(...)`):
 - The first argument is a `JFrame` parent. For now, we set it to null.
 - The second argument is the title of the Dialog box.
 - The last argument is its modality. We set it to *false*, for now.



Creating

- Creating

- The
- W
- th

- The new

JDialog

- The
- The
- The

```
private static void createJDialog() {
    JDialog dialog = new JDialog((JFrame) null, "This is a JDialog", false);
    dialog.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);

    JPanel panel = new JPanel();

    JLabel informationLabel = new JLabel("This is some information!");
    JButton okayButton = new JButton("Okay");

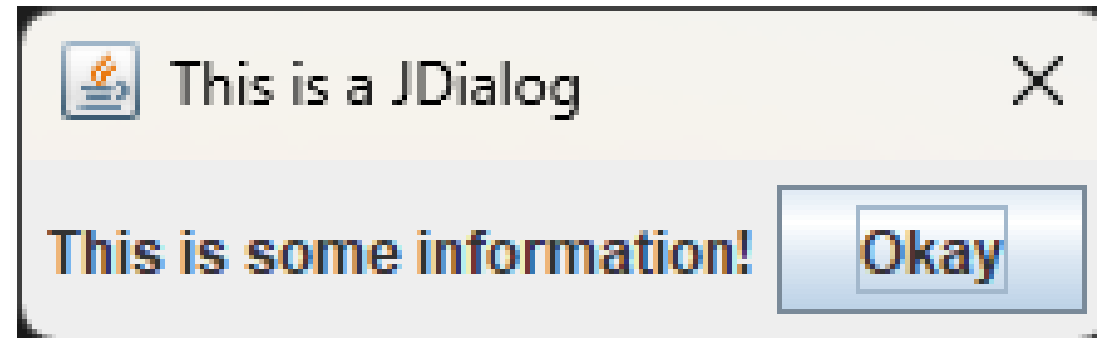
    panel.add(informationLabel);
    panel.add(okayButton);

    okayButton.addActionListener(new ActionListener(){
        public void actionPerformed(ActionEvent e) {
            dialog.dispose();
        }
    });

    dialog.add(panel);
    dialog.pack();
    dialog.setLocationRelativeTo(null);
    dialog.setVisible(true);
}
```



Result of previous code



Understanding the `JDialog` constructor

- The first argument is the parent of `JDialog`.
- `JDialog` isn't meant to be used as the main window of an application. It's meant to be used with a main `JFrame` window.
- When we create a `JDialog` we typically give it the reference to our main `JFrame`. So, when the dialog gets displayed, it gets displayed on top of our main Window.
- The second argument is the title of the window.
- The last argument is the *modality*. If we set it to true, it is going to pause execution after the `dialog.setVisible(true)` line and disable input to any other windows below it. It will resume the execution when the dialog is disposed.

```
JDialog dialog = new JDialog((JFrame) null, "This is a JDialog", false);
```

Setting modal to false

- Setting modal to *false* doesn't pause execution.
- Review the following code:
 - It will show the dialog box and display the print statement.

```
public static void main(String[] args) {  
    createJDialog();  
    System.out.println("This is output after creating the Dialog!");  
}
```

Setting modal to true

- Setting *modal* to “true”:

- Pauses execution after the `setVisible(true)` statement. Any line after that statement will not be executed.
- If multiple windows or dialog boxes exist in the program, it will only be able to detect input for the current dialog box.
- Until the dialog is disposed of, no input on any other window that belongs to program will be accepted.

```
public static void main(String[] args) {  
    createJDialog();  
    System.out.println("This is output after creating the Dialog!");  
}
```

● Making the function more useful

- • We can “*refactor*” `createJDialogFunction()` to create dialogs more flexibly.



Mak

- We
dia

```
public class App {
    public static void main(String[] args) {
        createDialog(null, "This is the Dialog title", "This is some information.", false);
    }

    private static void createDialog(JFrame parent, String title, String information, boolean modal) {
        Dialog dialog = new Dialog(parent, title, modal);
        dialog.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);

        JPanel panel = new JPanel();
        JLabel informationLabel = new JLabel(information);
        JButton okayButton = new JButton("Okay");

        panel.add(informationLabel);
        panel.add(okayButton);

        okayButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                dialog.dispose();
            }
        });

        dialog.add(panel);
        dialog.pack();
        dialog.setLocationRelativeTo(null);
        dialog.setVisible(true);
    }
}
```



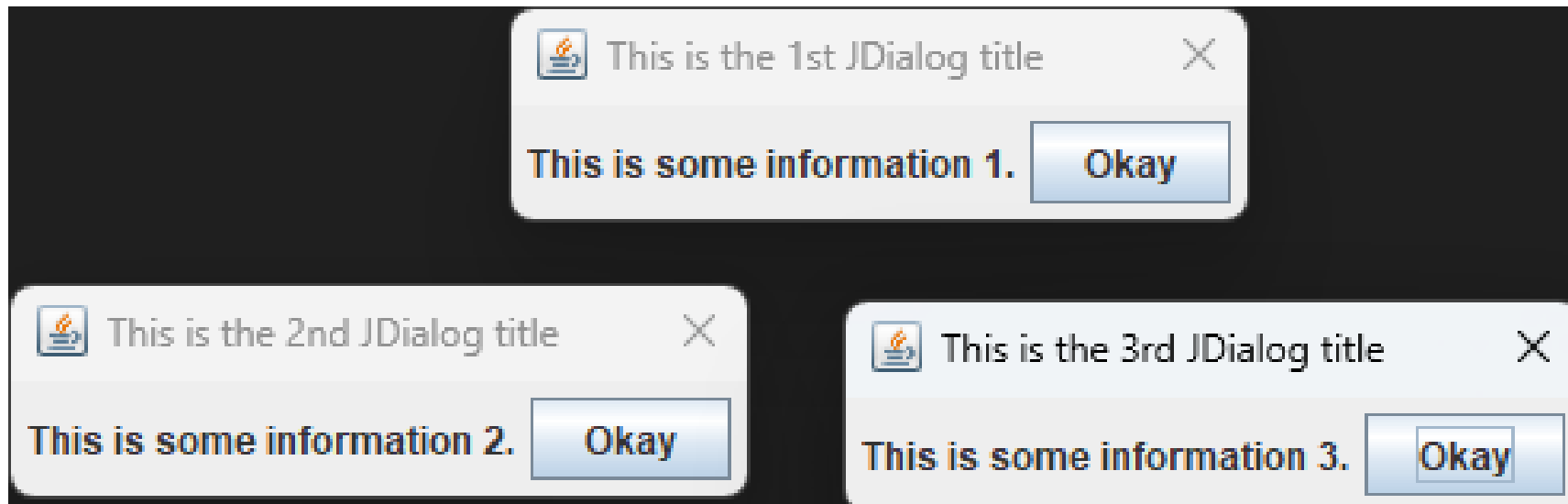
Multiple Dialog Boxes

- Displaying 3 dialog boxes simultaneously:
 - All of them are non-modal (modal = false).
 - They will all be centred the user has to move them around in order to see them.

```
public static void main(String[] args) {  
    createJDialog(null, "This is the 1st JDialog title", "This is some information 1.", false);  
    createJDialog(null, "This is the 2nd JDialog title", "This is some information 2.", false);  
    createJDialog(null, "This is the 3rd JDialog title", "This is some information 3.", false);  
}
```

Output of the previous code

- Setting *modal* to “true” for all the boxes:
 - Only the first is displayed.
 - When the first one closes, the second one will be displayed.
 - When the second one closes, the third one will be displayed.



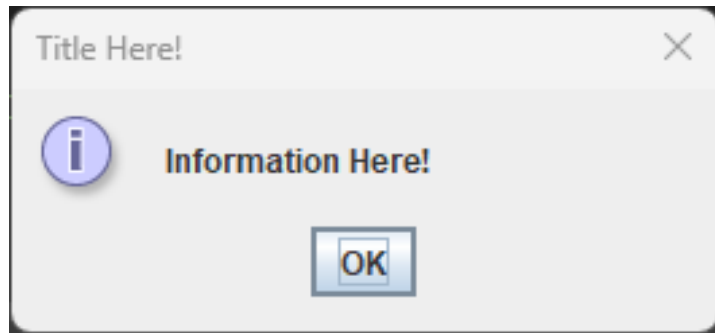
JOptionPane



JOptionPane

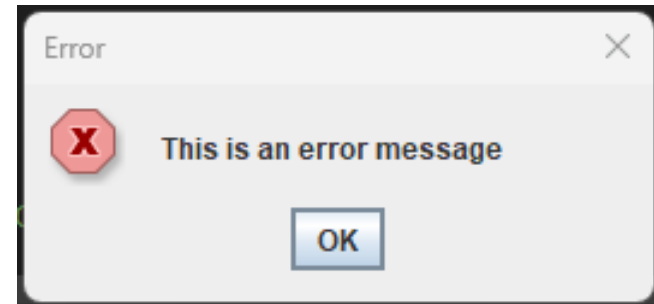
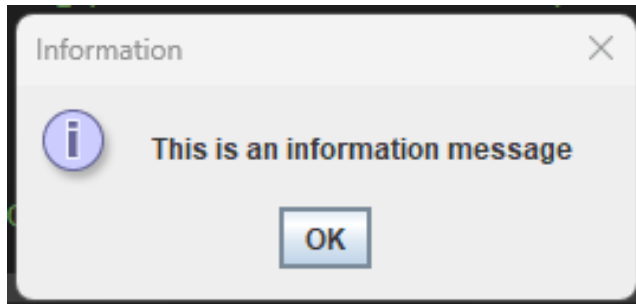
- JOptionPane essentially does all of this and packages it into a class.
- It also provides some decoration and primitives.
- For example, you can create a JDialog box easily by doing the following. The first argument is the parent JFrame.

```
JOptionPane.showMessageDialog(null, "Information Here!", "Title Here!", JOptionPane.INFORMATION_MESSAGE);
```



Pre-built message dialogs

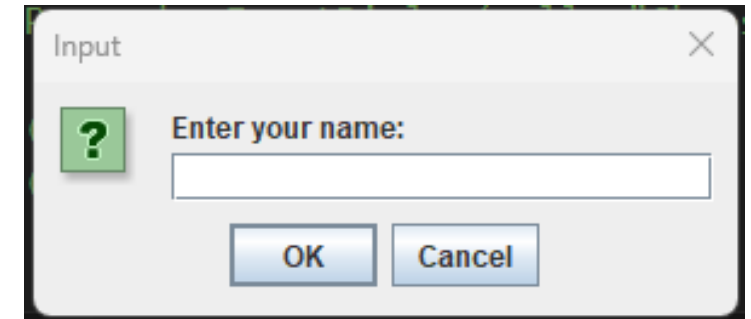
```
JOptionPane.showMessageDialog(null, "This is an information message", "Information", JOptionPane.INFORMATION_MESSAGE);  
JOptionPane.showMessageDialog(null, "This is a warning message", "Warning", JOptionPane.WARNING_MESSAGE);  
JOptionPane.showMessageDialog(null, "This is an error message", "Error", JOptionPane.ERROR_MESSAGE);
```



Pre-built Text input dialogs

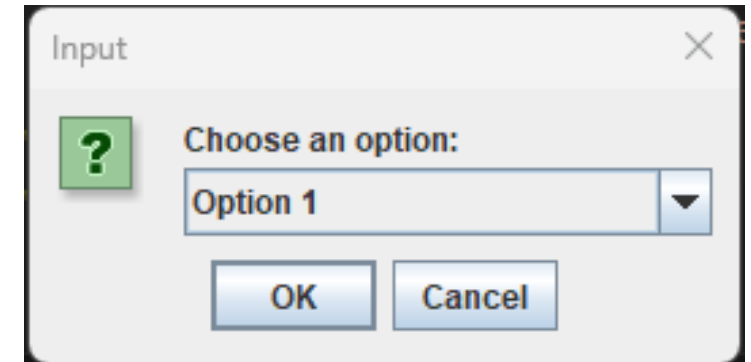
- ```
String userInput = JOptionPane.showInputDialog(null, "Enter your name:", "Input", JOptionPane.QUESTION_MESSAGE);
```

This will show an input dialog and when you press okay in the dialog, it will store the text input into the `userInput` string.



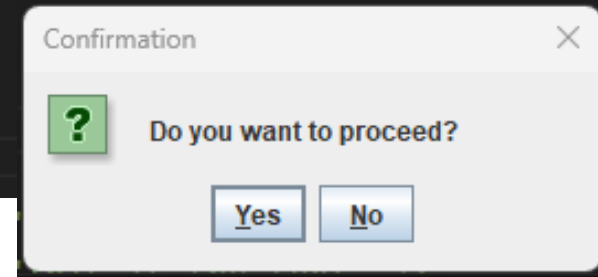
- ```
String[] options = {"Option 1", "Option 2", "Option 3"};  
String selectedOption = (String) JOptionPane.showInputDialog(null, "Choose an option:", "Input", JOptionPane.QUESTION_MESSAGE, null, options, options[0]);
```

This will show an input dialog and give you a list of options. The list of options are given as a string array, and the picked option is stored in the `selectedOption` string.

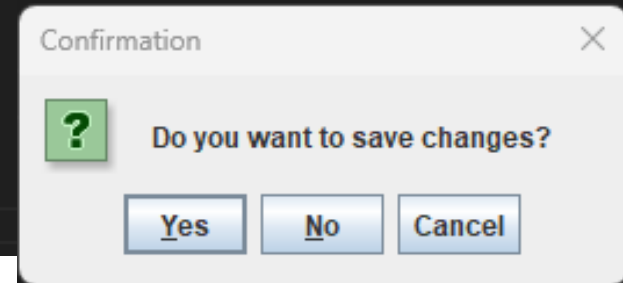


Pre-built confirm dialogs

- ```
int userChoice = JOptionPane.showConfirmDialog(null, "Do you want to proceed?", "Confirmation", JOptionPane.YES_NO_OPTION);
if (userChoice == JOptionPane.YES_OPTION) {
 // User clicked Yes
} else {
 // User clicked No
}
```



- ```
int userChoice = JOptionPane.showConfirmDialog(null, "Do you want to save changes?", "Confirmation", JOptionPane.YES_NO_CANCEL_OPTION);
if (userChoice == JOptionPane.YES_OPTION) {
    // User clicked Yes
} else if (userChoice == JOptionPane.NO_OPTION) {
    // User clicked No
} else {
    // User clicked Cancel
}
```



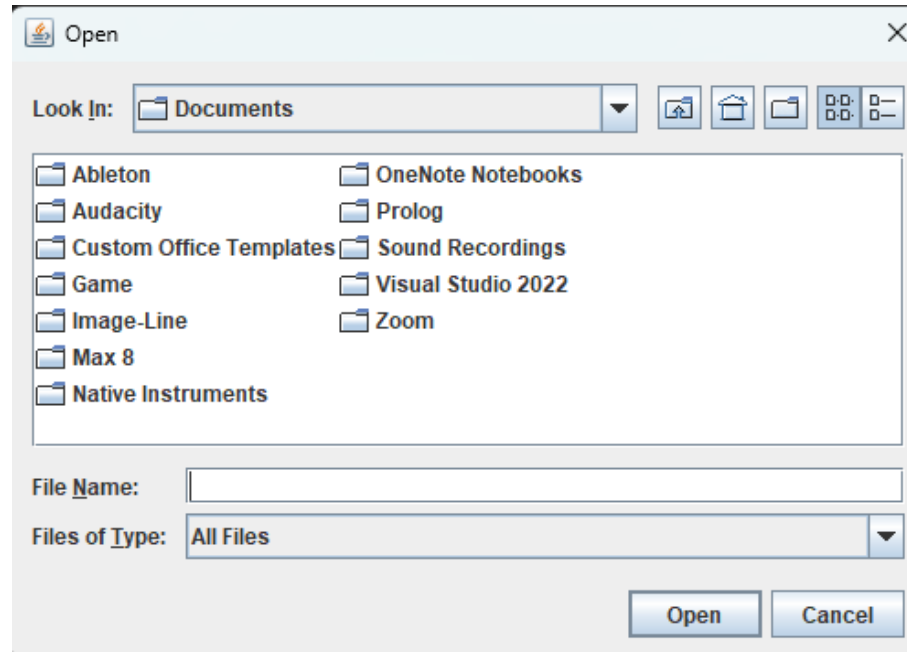


JFileChooser



JFileChooser

- JFileChooser is a class that easily allows a user to select a file from the computer using a simple GUI. The dialog that opens is a relatively sophisticated JDialog.



Showing the file chooser

1. Create a `JFileChooser` object
2. Call the `showOpenDialog()` function.
The parameter it takes in is the parent of the `JDialog` that gets opened.
The functionality of the parent was discussed before. Refer to previous slides.

```
public static void main(String[] args) {  
    JFileChooser chooser = new JFileChooser();  
    chooser.showOpenDialog(null);  
}
```

- Getting the selected file.

```
public static void main(String[] args) {  
    JFileChooser chooser = new JFileChooser();  
    int result = chooser.showOpenDialog(null);  
  
    File selectedFile = null;  
  
    if (result == JFileChooser.APPROVE_OPTION) {  
        selectedFile = chooser.getSelectedFile();  
        System.out.println("Selected file: " + selectedFile.getName());  
    } else {  
        System.out.println("File selection canceled.");  
    }  
}
```



Useful Resources





Useful Resources

- **JDialog and JOptionPane:**

- <https://docs.oracle.com/javase%2Ftutorial%2Fswing%2F%2F/components/dialog.html>

- **JFileChooser:**

- <https://docs.oracle.com/javase%2F7%2Fdocs%2Fapi%2F%2F/javafx/swing/JFileChooser.html>



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor